SIMATS ENGINEERING

COURSE CODE: CSA0311

COURSE NAME: DATA STRUCTURES

C04-ASSESSMENT TOOL 1 (High)- Comparative Analysis

S.No	REG NO	NAME	ASSESSMENT TOOL 1 (High)- Comparative Analysis
1	192210722	ADABALA ADINARAYANA	A system needs to store large data and perform frequent search operations. For this, compare Hashing and Sorting-based search (Binary Search) in terms of: a) Time complexity b) Space usage c) Update operations d) Real-world applicability Conclude which is better for dynamic systems. Compare Quick Sort and Merge Sort based on: a) Best, average, and worst-case complexity b) Stability c) Memory usage d) Practical performance Explain why Quick Sort is often preferred in real applications despite its worst-case complexity.
2	192511354	Akileshwaran A	A system needs to repeatedly: a) Insert elements b) Retrieve the highest priority element Compare Heap and Sorted Array in terms of: a) Insertion time b) Deletion time c) Efficiency in real-time systems Suggest which structure is better for scheduling systems. Compare Linear Probing and Separate Chaining for collision handling

			in hashing based on: a) Clustering effect b) Memory usage c) Performance under high load factor Which method is more suitable for large-scale applications and why?
3	192511484	Alan Philip	Compare Sequential Sorting and Parallel Sorting (Multi-threading / MapReduce) in terms of: a) Execution time b) Scalability c) Overhead d) Suitability for small vs large datasets Explain why parallel sorting is not always the best choice. Compare separate chaining versus open addressing (linear probing) for a hash table storing 10 million key-value pairs with a load factor of 0.75. Analyze memory usage, cache performance, worst-case lookup time, and deletion complexity. Under what scenarios would you choose one over the other?
4	192511299	Anlo Melwin S	Compare linear probing, quadratic probing, and double hashing as collision resolution strategies. Analyze primary clustering, secondary clustering, and average probe counts for successful and unsuccessful searches. Which method offers the best cache performance and why? Compare a binary heap (priority queue) versus a balanced binary search tree (e.g., AVL or Red-Black) for implementing a scheduler that requires both min extraction and arbitrary key updates. Analyze time complexity for insert, extract-min, and decrease-key operations. Which data structure is more memory-efficient?
5	192525115	Chinna Thumbalam Mohammed Ubed	Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? Compare heap sort versus quick sort for sorting 1 billion integers in a real-time system with strict latency requirements (no operation can exceed 1 second). Analyze worst-case time complexity, space complexity, and predictability. Which algorithm guarantees O(n log n) worst-case and why would you choose it despite quick sort's faster average case?

6	192524424	Dhineshkumar M C	Compare radix sort (LSD, base 256) versus comparison-based sorts (quick sort, merge sort) for sorting 100 million 32-bit integers. Analyze time complexity in terms of O(d × n) vs. O(n log n), memory usage (counting array), and practical performance on modern CPUs. When does radix sort lose its advantage? Compare insertion sort versus quick sort for sorting small arrays (size < 50). Analyze constant factors, branch prediction, cache locality, and overhead of recursive calls. Why do production sorting implementations (e.g., Timsort, Introsort) switch to insertion sort for small subarrays?
7	192511450	Finny Franklin R	Compare single-threaded quick sort, parallel quick sort with 8 threads, and parallel merge sort with 8 threads on a shared-memory multi-core system. Analyze theoretical speedup (Amdahl's law), overhead of synchronization, load imbalance, and memory bandwidth contention. Which algorithm scales better and why? Compare MapReduce sorting (total order partitioning) versus parallel merge sort on a shared-memory machine for sorting 10 TB of data distributed across 100 nodes. Analyze network communication cost, fault tolerance, disk I/O, and scalability. Under what conditions does MapReduce become necessary despite its higher overhead?
8	192425187	JAGALETI SANTHOSH GOUD	Compare GPU bitonic sort (CUDA) versus CPU parallel quick sort (64 cores) for sorting 1 billion integers. Analyze parallel time complexity ($O(\log^2 n)$ vs. $O((n \log n)/p)$), memory hierarchy (global vs. shared memory), warp divergence, and PCIe transfer overhead. For what input sizes does GPU sorting become advantageous? Compare linear probing and cuckoo hashing for a read-heavy workload (90% lookups, 10% inserts) with 100 million keys. Analyze worst-case lookup time guarantees, insertion failures, rehashing costs, and memory utilization. Which hash table variant offers better predictable latency?
9	192521353	Jai Akash P	Compare binary heaps and Fibonacci heaps for implementing Dijkstra's shortest path algorithm on a dense graph with 100,000 nodes and 10 million edges. Analyze amortized time complexity for decrease-key operations, memory overhead, and practical performance. Why is Fibonacci heap rarely used in practice despite better theoretical bounds? Compare Timsort (adaptive hybrid sort) versus traditional merge sort for sorting real-world data with the following characteristics: (a) already sorted array, (b) reverse sorted array, (c) array with many duplicate keys, (d) array with random order. Analyze best-case, average-case, and worst-case time complexity for each scenario.

10	192572094	K Amardeep	Compare hash table with separate chaining versus binary search tree for implementing a dictionary in a multi-threaded environment. Analyze locking granularity, concurrent access patterns, resizing complexity, and cache behavior. Which structure supports more scalable concurrent operations? Compare counting sort, radix sort, and bucket sort for sorting floating-point numbers in the range [0.0, 1.0) with uniform distribution. Analyze how each algorithm handles non-integer keys, distribution sensitivity, memory usage, and stability. Which algorithm is most suitable for sorting 100 million uniformly distributed floats and why?
11	192525307	Karnati Nikhil	Compare counting sort, radix sort, and bucket sort for sorting floating-point numbers in the range [0.0, 1.0) with uniform distribution. Analyze how each algorithm handles non-integer keys, distribution sensitivity, memory usage, and stability. Which algorithm is most suitable for sorting 100 million uniformly distributed floats and why? Compare Quadratic Probing and Double Hashing in terms of: a) Clustering issues b) Collision resolution efficiency c) Suitability for dense hash tables
12	192321191L	KARTHICK S	Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: a) Practical significance b) Impact on algorithm selection Compare Recursive and Iterative implementations of Quick Sort in terms of: a) Space complexity (stack usage) b) Ease of implementation c) Risk of stack overflow
13	192511232	Kiruthika B	Compare Merge Sort and Radix Sort based on the following: a) Type of sorting technique b) Input limitations c) Performance on large datasets Compare two methods of building a heap: a) Bottom-up heap construction b) Repeated insertion into heap Which is more efficient and why?

14	192572077	Konetigari Tharlika	Compare Heap-based priority queue and array-based priority queue in terms of: a) Insert operation b) Delete operation c) Overall efficiency Compare Binary Heap and Binary Search Tree (BST) based on the following: a) Searching capability b) Insert/delete operations c) Typical applications
15	192525173	Kunasi Yoganandh	Compare Internal sorting and External sorting based on the following: a) Memory usage b) Data size handled c) Performance considerations Compare Parallel Merge Sort and Sequential Merge Sort in terms of: a) Execution time b) Overhead due to parallelism Suitability for large datasets
16	192521226	Lingeshwar T	Compare Single-core (sequential) sorting and multi-core sorting based on the following: a) Performance improvement b) Synchronization overhead c) Practical challenges Compare Time complexity and Space complexity in terms of: a) Trade-offs b) Situations where one is prioritized over the other
17	192521332	Madhu Mitha V	Compare Direct Addressing and Hashing based on the following: a) Memory requirements b) Flexibility c) Real-world applicability Compare the effect of load factor in hash tables with input size in sorting algorithms based on the following: a) Performance degradation b) Optimization strategies

18	192524372	Mahalakshmi S	Compare Heap Sort and Quick Sort in terms of: a) Worst-case guarantees b) Practical performance c) Use cases Compare Radix Sort and Counting Sort based on the following: a) Working mechanism b) Range of input values c) Efficiency
19	192521137	Mekala Vineeth Kumar	Compare Single-threaded sorting and distributed sorting (e.g., cluster-based) based on the following: a) Scalability b) Resource requirements c) Complexity of implementation Compare Binary Heap and Fibonacci Heap in terms of: a) Insert operation b) Decrease-key operation c) Practical usability in real systems
20	192525108	M Harsha	Compare two approaches for searching based on the following: a) Sorting data first and then applying binary search b) Using hashing for direct lookup Compare two approaches for sorting based on their implementation and write it's complexities: a) Quick sort and merge sort b) Insertion and Radix sort
21	192372146	MALLURU VENKATA SAKETH	A system needs to store large data and perform frequent search operations. For this, compare Hashing and Sorting-based search (Binary Search) in terms of: e) Time complexity f) Space usage g) Update operations h) Real-world applicability Conclude which is better for dynamic systems. Compare Quick Sort and Merge Sort based on: e) Best, average, and worst-case complexity f) Stability g) Memory usage h) Practical performance

			Explain why Quick Sort is often preferred in real applications despite its worst-case complexity.
22	192525413	Mokshitha Kilari	A system needs to repeatedly: c) Insert elements d) Retrieve the highest priority element Compare Heap and Sorted Array in terms of: d) Insertion time e) Deletion time f) Efficiency in real-time systems Suggest which structure is better for scheduling systems. Compare Linear Probing and Separate Chaining for collision handling in hashing based on: d) Clustering effect e) Memory usage f) Performance under high load factor Which method is more suitable for large-scale applications and why?
23	192524315	Monica Sentamil K	Compare Sequential Sorting and Parallel Sorting (Multi-threading / MapReduce) in terms of: e) Execution time f) Scalability g) Overhead h) Suitability for small vs large datasets Explain why parallel sorting is not always the best choice. Compare separate chaining versus open addressing (linear probing) for a hash table storing 10 million key-value pairs with a load factor of 0.75. Analyze memory usage, cache performance, worst-case lookup time, and deletion complexity. Under what scenarios would you choose one over the other?
24	192511353	Mosikiran K	Compare linear probing, quadratic probing, and double hashing as collision resolution strategies. Analyze primary clustering, secondary clustering, and average probe counts for successful and unsuccessful searches. Which method offers the best cache performance and why? Compare a binary heap (priority queue) versus a balanced binary search tree (e.g., AVL or Red-Black) for implementing a scheduler that requires both min extraction and arbitrary key updates. Analyze time complexity for insert, extract-min, and decrease-key operations.

			Which data structure is more memory-efficient?
25	192524090	Murali Krishnan N	Compare quick sort and merge sort for sorting a large linked list of 10 million nodes. Analyze memory requirements (in-place vs. extra space), cache behavior, and time complexity. Which algorithm is preferable for linked lists and why? How does the answer change for arrays? Compare heap sort versus quick sort for sorting 1 billion integers in a real-time system with strict latency requirements (no operation can exceed 1 second). Analyze worst-case time complexity, space complexity, and predictability. Which algorithm guarantees O(n log n) worst-case and why would you choose it despite quick sort's faster average case?
26	192424240	NASINA UMASRI	Compare radix sort (LSD, base 256) versus comparison-based sorts (quick sort, merge sort) for sorting 100 million 32-bit integers. Analyze time complexity in terms of O(d × n) vs. O(n log n), memory usage (counting array), and practical performance on modern CPUs. When does radix sort lose its advantage? Compare insertion sort versus quick sort for sorting small arrays (size < 50). Analyze constant factors, branch prediction, cache locality, and overhead of recursive calls. Why do production sorting implementations (e.g., Timsort, Introsort) switch to insertion sort for small subarrays?
27	192511311	Natesa Krishnan V	Compare single-threaded quick sort, parallel quick sort with 8 threads, and parallel merge sort with 8 threads on a shared-memory multi-core system. Analyze theoretical speedup (Amdahl's law), overhead of synchronization, load imbalance, and memory bandwidth contention. Which algorithm scales better and why? Compare MapReduce sorting (total order partitioning) versus parallel merge sort on a shared-memory machine for sorting 10 TB of data distributed across 100 nodes. Analyze network communication cost, fault tolerance, disk I/O, and scalability. Under what conditions does MapReduce become necessary despite its higher overhead?
28	192572082	Nithiyasri S	Compare GPU bitonic sort (CUDA) versus CPU parallel quick sort (64 cores) for sorting 1 billion integers. Analyze parallel time complexity ($O(\log^2 n)$ vs. $O((n \log n)/p)$), memory hierarchy (global vs. shared memory), warp divergence, and PCIe transfer overhead. For what input sizes does GPU sorting become advantageous? Compare linear probing and cuckoo hashing for a read-heavy workload (90% lookups, 10% inserts) with 100 million keys. Analyze worst-case lookup time guarantees, insertion failures, rehashing costs,

			and memory utilization. Which hash table variant offers better predictable latency?
29	192525111	Peddasani Sai Ganesh Reddy	Compare binary heaps and Fibonacci heaps for implementing Dijkstra's shortest path algorithm on a dense graph with 100,000 nodes and 10 million edges. Analyze amortized time complexity for decrease-key operations, memory overhead, and practical performance. Why is Fibonacci heap rarely used in practice despite better theoretical bounds? Compare Timsort (adaptive hybrid sort) versus traditional merge sort for sorting real-world data with the following characteristics: (a) already sorted array, (b) reverse sorted array, (c) array with many duplicate keys, (d) array with random order. Analyze best-case, average-case, and worst-case time complexity for each scenario.
30	192472054	PESALA SUMANTH KUMAR	Compare hash table with separate chaining versus binary search tree for implementing a dictionary in a multi-threaded environment. Analyze locking granularity, concurrent access patterns, resizing complexity, and cache behavior. Which structure supports more scalable concurrent operations? Compare counting sort, radix sort, and bucket sort for sorting floating-point numbers in the range [0.0, 1.0) with uniform distribution. Analyze how each algorithm handles non-integer keys, distribution sensitivity, memory usage, and stability. Which algorithm is most suitable for sorting 100 million uniformly distributed floats and why?
31	192521362	Rahul R	Compare counting sort, radix sort, and bucket sort for sorting floating-point numbers in the range [0.0, 1.0) with uniform distribution. Analyze how each algorithm handles non-integer keys, distribution sensitivity, memory usage, and stability. Which algorithm is most suitable for sorting 100 million uniformly distributed floats and why? Compare Quadratic Probing and Double Hashing in terms of: d) Clustering issues e) Collision resolution efficiency f) Suitability for dense hash tables
32	192421285	RANJITHA R	Compare Best-case and Worst-case time complexity of sorting algorithms based on the following: c) Practical significance d) Impact on algorithm selection Compare Recursive and Iterative implementations of Quick Sort in terms of:

			d) Space complexity (stack usage) e) Ease of implementation f) Risk of stack overflow
33	192421276	ROHITH S	Compare Merge Sort and Radix Sort based on the following: d) Type of sorting technique e) Input limitations f) Performance on large datasets Compare two methods of building a heap: c) Bottom-up heap construction d) Repeated insertion into heap Which is more efficient and why?
34	192525361	Shiak Rihan	Compare Heap-based priority queue and array-based priority queue in terms of: d) Insert operation e) Delete operation f) Overall efficiency Compare Binary Heap and Binary Search Tree (BST) based on the following: d) Searching capability e) Insert/delete operations f) Typical applications
35	192511408	Sweta R	Compare Internal sorting and External sorting based on the following: d) Memory usage e) Data size handled f) Performance considerations Compare Parallel Merge Sort and Sequential Merge Sort in terms of: c) Execution time d) Overhead due to parallelism Suitability for large datasets
36	192565006	Thanigaivel S	Compare Single-core (sequential) sorting and multi-core sorting based on the following: d) Performance improvement e) Synchronization overhead f) Practical challenges Compare Time complexity and Space complexity in terms of:

			c) Trade-offs d) Situations where one is prioritized over the other
37	192524399	Tharun D	Compare Direct Addressing and Hashing based on the following: d) Memory requirements e) Flexibility f) Real-world applicability Compare the effect of load factor in hash tables with input size in sorting algorithms based on the following: c) Performance degradation d) Optimization strategies
38	192524440	Vaishali M	Compare Heap Sort and Quick Sort in terms of: d) Worst-case guarantees e) Practical performance f) Use cases Compare Radix Sort and Counting Sort based on the following: d) Working mechanism e) Range of input values f) Efficiency
39	192525213	V Deepa Dharshini	Compare Single-threaded sorting and distributed sorting (e.g., cluster-based) based on the following: d) Scalability e) Resource requirements f) Complexity of implementation Compare Binary Heap and Fibonacci Heap in terms of: d) Insert operation e) Decrease-key operation f) Practical usability in real systems
40	192511177	Vignesh P S	Compare two approaches for searching based on the following: c) Sorting data first and then applying binary search d) Using hashing for direct lookup Compare two approaches for sorting based on their implementation and write it's complexities: c) Quick sort and merge sort d) Insertion and Radix sort